# GeoFinance Lens

Technical Deep Dive Document

Parth Gopakumar
Comp B
Roll No. 1022226

April 9, 2026

## Contents

**1 Project Summary** **4**

**2 Core Product Goal** **4**

**3 Theme Alignment** **4**

**4 Prototype-First Stack** **5**

4.1 Recommended Core Stack . . . . . 5

4.2 Why This Stack . . . . . 5

**5 Free Live News Source for the Prototype** **6**

5.1 Primary News API . . . . . 6

5.2 Fallback Source . . . . . 6

5.3 News Selection Rules . . . . . 6

**6 High-Level Architecture** **7**

6.1 System Diagram . . . . . 7

6.2 What Each Layer Does . . . . . 8

**7 Libraries and Packages** **8**

7.1 Frontend Packages . . . . . 8

7.2 Backend Packages . . . 8
7.3 Cloud and AI Tools . . . 9

**8 Detailed Data Model** **9**
8.1 Raw News Article Shape . . . 9
8.2 Normalized Event Shape . . . 10
8.3 UI State Shape . . . 10

**9 Backend API Design** **11**
9.1 Route List . . . 11
9.2 Route Behavior . . . 11

**10 Classification Logic** **12**
10.1 Core Buckets . . . 12
10.2 Simple Rule Strategy . . . 12

**11 Gemma Integration Contract** **13**
11.1 What Gemma Must Do . . . 13
11.2 Gemma Prompt Format . . . 13
11.3 Gemma Output Format . . . 13
11.4 Gemma Guardrails . . . 14

**12 Map and Visualization Design** **14**
12.1 Map Requirements . . . 14
12.2 Marker Rules . . . 14
12.3 UI Panels . . . 15

**13 Codex Build Contract** **15**
13.1 What Codex Must Understand . . . 15
13.2 What Codex Should Generate First . . . 16
13.3 Codex Rules . . . 16
13.4 What Codex Should Not Do . . . 17

**14 Recommended File Structure** **17**

**15 Environment Variables** **18**

**16 Implementation Flow** **19**

16.1 User Journey . . . . . 19

16.2 Event Processing Flow . . . . . 19

**17 Testing Strategy** **20**

17.1 Unit Tests . . . . . 20

17.2 Integration Tests . . . . . 20

17.3 Smoke Tests . . . . . 20

**18 Deployment Plan** **21**

18.1 Local Development . . . . . 21

18.2 Cloud Deployment . . . . . 21

18.3 Demo Safety Rules . . . . . 21

**19 Risks and Mitigation** **21**

**20 Deliverables** **22**

**21 Conclusion** **22**

# 1 Project Summary

GeoFinance Lens is a map-first news intelligence web application that converts breaking headlines into simple, structured financial meaning. It scans live news, filters for geopolitics, Indian politics, inflation, oil, policy, and other market-moving incidents, then explains what the event means for people, markets, and personal finances.

The product is not a stock prediction engine. It is an interpretation layer over news.

# 2 Core Product Goal

The app must answer these six questions for every event:

1. What happened?
2. Why does it matter?
3. Where did it happen?
4. What may happen next?
5. How can it affect markets?
6. How can it affect a normal person's money and spending?

The final output must always include a **personal finance impact** and a **suggested action**. That is what makes the project fit the track.

# 3 Theme Alignment

The hackathon track is **Money & Markets for Everyone**. This project fits because it turns complex headlines into easy language and connects those headlines to:

- daily expenses,
- fuel and transport costs,
- inflation pressure,
- EMI and loan sensitivity,
- savings behavior,
- and purchasing decisions.

The strongest positioning is not "news summarization." The strongest positioning is "decision-ready financial context from live news."

# 4 Prototype-First Stack

This project must be built so that the prototype works even with limited time, limited money, and API failures.

## 4.1 Recommended Core Stack

- **Frontend**: React
- **Build Tool**: Vite
- **Styling**: Tailwind CSS
- **Map Layer**: Leaflet
- **State Management**: Zustand
- **HTTP Client**: Axios
- **Backend**: Node.js + Express
- **Config**: dotenv
- **Cross-Origin Support**: cors
- **Validation**: zod
- **Testing**: Jest + Supertest
- **AI Reasoning**: Gemma on Vertex AI
- **Hosting**: Cloud Run
- **Database**: Firestore

## 4.2 Why This Stack

The stack is selected for speed, reliability, and demo safety:

- React gives component-based UI building.
- Vite gives fast local development and production bundling.
- Tailwind allows rapid styling without writing heavy CSS files.
- Leaflet gives a free, lightweight, interactive map experience.
- Zustand keeps state simple and avoids Redux complexity.
- Axios simplifies backend and API calls.

- Express makes the API structure clean and easy for Codex to generate.
- dotenv keeps secrets and API keys out of source code.
- cors avoids browser blocking during local development.
- zod keeps incoming data safe and structured.
- Jest and Supertest let the core logic be verified quickly.
- Cloud Run supports containerized deployment without infrastructure work.
- Firestore stores event data, history, and demo content.
- Gemma creates the explanation and impact layer.

# 5 Free Live News Source for the Prototype

## 5.1 Primary News API

Use **GNews** as the primary live news source for the prototype. The free plan is suitable for a hackathon demo because it provides:

- 100 requests per day,
- up to 10 articles per request,
- localhost CORS support,
- and a 12-hour delay on the free plan.

## 5.2 Fallback Source

If the live API fails, the backend must immediately switch to a local JSON dataset.

The app must never depend only on live news, because hackathon demos must be stable.

## 5.3 News Selection Rules

Not every headline should be shown. The system should keep only headlines that match:

- geopolitics,
- Indian politics,
- RBI or policy movement,
- oil, energy, or shipping,

- inflation or macroeconomics,
- conflict or sanctions,
- trade or supply-chain events.

# 6 High-Level Architecture

## 6.1 System Diagram

```
User
  |
  v
React UI (Stitch-designed layout)
  |
  v
Express API on Node.js
  |
  +-----------------------------+
  |                             |
  v                             v
GNews Live API              Local JSON Backup
  |                             |
  +--------------+--------------+
                 |
                 v
        News Normalizer Layer
                 |
                 v
     Category + Region Classifier
                 |
                 v
      Coordinate Mapper / Hotspot Engine
                 |
                 v
      Gemma Reasoning Layer on Vertex AI
                 |
                 v
  Impact Output: explanation + market effect + personal finance effect + action
                 |
                 v
     UI cards + map markers + detail panel
```

### 6.2 What Each Layer Does

**1. React UI** Displays the dashboard, filters, map, event list, and explanation panel.

**2. Express API** Fetches news, filters data, normalizes events, and serves structured JSON.

**3. News Normalizer Layer** Converts raw article objects into a clean internal event format.

**4. Classifier Layer** Detects whether a headline belongs to geopolitics, Indian politics, oil, inflation, or another relevant bucket.

**5. Coordinate Mapper** Assigns latitude and longitude based on the region.

**6. Gemma Reasoning Layer** Generates the simple explanation, impact breakdown, and suggested action.

**7. Firestore** Stores event history, fallback events, user preferences, and demo logs.

**8. Cloud Run** Hosts the backend API and optionally the frontend container.

## 7 Libraries and Packages

### 7.1 Frontend Packages

| Library | Role | Why It Is Used |
|---|---|---|
| React | UI framework | Builds the app from reusable components such as map, cards, filters, and panels. |
| Vite | Build tool | Gives fast startup and easy local development. |
| Tailwind CSS | Styling | Makes the dashboard look polished quickly. |
| Leaflet | Map engine | Provides a free interactive world map with zoom, pan, and markers. |
| Zustand | State management | Keeps selected event, filters, and map state simple. |
| Axios | HTTP client | Sends requests to backend routes and external APIs. |
| date-fns | Date formatting | Formats timestamps, relative dates, and event freshness. |
| lucide-react | Icons | Adds lightweight icons for actions and status indicators. |

### 7.2 Backend Packages

| Library | Role | Why It Is Used |
|---|---|---|
| Node.js | Runtime | Runs the API server and business logic. |
| Express | API framework | Handles routes, middleware, and JSON responses. |
| cors | Middleware | Allows browser requests from the frontend during development. |
| dotenv | Config loader | Loads API keys and environment values from .env. |
| zod | Validation | Ensures article and event objects follow the expected schema. |
| Jest | Testing | Tests pure functions and classification logic. |
| Supertest | API testing | Tests API routes without manual browser clicks. |

### 7.3 Cloud and AI Tools

- **Gemma on Vertex AI**: explanation generation, financial impact reasoning, suggested action generation.
- **Cloud Run**: container deployment for backend and optionally frontend.
- **Firestore**: event storage, cached scans, fallback data, and demo state.

## 8 Detailed Data Model

### 8.1 Raw News Article Shape

```
{
  "title": "RBI keeps repo rate unchanged",
  "description": "Policy update from the central bank.",
  "content": "Short content from API or fallback dataset",
  "url": "https://...",
  "image": "https://...",
  "publishedAt": "2026-04-09T00:00:00Z",
  "source": {
    "name": "News source name",
    "url": "https://..."
  }
}
```

### 8.2 Normalized Event Shape

```
{
  "id": "evt_001",
  "title": "RBI keeps repo rate unchanged",
  "category": "India / Economy",
  "subCategory": "Monetary Policy",
  "region": "India",
  "lat": 20.5937,
  "lng": 78.9629,
  "priority": "high",
  "summary": "The central bank kept interest rates steady.",
  "whatHappened": "Short plain-language explanation.",
  "whyItMatters": "Why this matters for markets and people.",
  "marketImpact": ["banks", "loan rates", "bond yields"],
  "personalFinanceImpact": [
    "EMIs may remain stable for now",
    "borrowers may not see immediate pressure"
  ],
  "suggestedAction": [
    "Review your loan plan",
    "Watch the next policy update"
  ],
  "possibleOutcome": [
    "Rates may stay stable in the short term"
  ],
  "source": "gnews",
  "publishedAt": "2026-04-09T00:00:00Z"
}
```

### 8.3 UI State Shape

```
{
  "selectedCategory": "all",
  "selectedRegion": "India",
  "explainMode": "normal",
  "selectedEventId": "evt_001",
  "events": [],
  "loading": false,
  "error": null,
  "mapCenter": [20.5937, 78.9629],
  "mapZoom": 2
```

```
}
```

# 9 Backend API Design

## 9.1 Route List

| Route | Method | Purpose |
|---|---|---|
| /api/health | GET | Confirms server is running. |
| /api/scan | GET | Fetches live or fallback headlines and returns normalized events. |
| /api/events | GET | Returns stored events and cached demo results. |
| /api/events/:id | GET | Returns one event in detail. |
| /api/fallback | GET | Returns local fallback JSON for demo reliability. |
| /api/config | GET | Returns safe UI config such as categories and map defaults. |

## 9.2 Route Behavior

### /api/scan

1. Call GNews.
2. Check response validity.
3. Filter irrelevant headlines.
4. Classify relevant headlines.
5. Attach regions and coordinates.
6. Send each event to Gemma.
7. Merge reasoning output into final event objects.
8. Return structured events to the frontend.

### /api/events

1. Read cached events from Firestore or memory.
2. Return the last successful scan.

**/api/fallback**

1. Load local JSON.
2. Return a stable dataset for demo mode.

# 10 Classification Logic

## 10.1 Core Buckets

The classifier should use keyword scoring rather than full ML training. That keeps the demo simple and predictable.

Main buckets:

- Indian politics
- RBI / monetary policy
- inflation / macroeconomics
- oil / energy
- conflict / war / sanctions
- trade / supply chain
- global markets

## 10.2 Simple Rule Strategy

Each article gets a score for each bucket. The highest score determines the category.

Example logic:

```
if title or description contains:
  "RBI", "repo rate", "inflation" -> India / Economy

if title or description contains:
  "war", "attack", "sanction", "missile" -> Geopolitics / Conflict

if title or description contains:
  "oil", "crude", "OPEC" -> Energy / Oil

if title or description contains:
  "election", "parliament", "government" -> Indian Politics
```

# 11 Gemma Integration Contract

## 11.1 What Gemma Must Do

Gemma is not the whole app. It is the reasoning layer.

Gemma must generate:

- a one-line plain-language summary,
- a "why it matters" explanation,
- a market impact summary,
- a personal finance impact summary,
- a suggested action,
- a possible short-term outcome.

## 11.2 Gemma Prompt Format

```
SYSTEM:
You are a financial news explanation engine.
Explain the event in simple language for students and non-experts.
Never predict exact prices.
Always include:
1. what happened
2. why it matters
3. market impact
4. personal finance impact
5. suggested action
6. possible outcome

USER:
Event title: ...
Event summary: ...
Category: ...
Region: ...
```

## 11.3 Gemma Output Format

Gemma should return valid JSON only:

```
{
  "whatHappened": "...",
  "whyItMatters": "...",
  "marketImpact": ["...", "..."],
  "personalFinanceImpact": ["...", "..."],
  "suggestedAction": ["...", "..."],
  "possibleOutcome": ["...", "..."]
}
```

### 11.4 Gemma Guardrails

- Do not generate stock tips.
- Do not promise future price movement.
- Do not use dense finance jargon.
- Do not return incomplete JSON.
- Do not omit personal finance impact.
- Do not omit suggested action.

## 12 Map and Visualization Design

### 12.1 Map Requirements

The map must be:

- zoomable,
- pannable,
- readable at global scale,
- readable at country scale,
- visually dark but not cluttered.

### 12.2 Marker Rules

Markers should:

- glow softly,
- pulse on active events,

- open a detail card on click,
- move the map to the event region,
- show priority with color or size.

### 12.3 UI Panels

The UI should have:

- top action bar,
- left or side headline list,
- central map,
- right detail panel,
- bottom impact/action summary for mobile.

## 13 Codex Build Contract

This section tells Codex exactly how to understand the project and what to generate.

### 13.1 What Codex Must Understand

Codex should read this document and understand:

- the product is a news-to-money interpretation app,
- the app must work even if the live API fails,
- every event must include personal finance impact,
- every event must include a suggested action,
- the map is central to the experience,
- the output must be simple enough for non-experts,
- the code must be split into frontend, backend, services, utils, and tests,
- secrets must never be hardcoded,
- the logic must be deterministic when AI fails.

### 13.2 What Codex Should Generate First

Codex should build in this order:

1. folder structure,
2. backend server,
3. news fetch service,
4. filtering and classification utilities,
5. coordinate mapper,
6. Gemma service wrapper,
7. frontend layout,
8. map integration,
9. event cards,
10. fallback data handling,
11. tests,
12. deployment configuration.

### 13.3 Codex Rules

Codex must follow these rules:

- Use pure functions for classification and formatting.
- Keep UI components small and reusable.
- Keep API handlers thin.
- Put business logic in services or utils, not in routes.
- Use environment variables for all API keys.
- Add fallback handling before adding advanced features.
- Add tests for every core pure function.
- Avoid unnecessary libraries.

### 13.4 What Codex Should Not Do

- Do not overcomplicate the data model.
- Do not build user authentication unless required later.
- Do not use paid APIs for the first prototype.
- Do not rely only on Gemma for all logic.
- Do not remove the fallback JSON mode.

## 14 Recommended File Structure

```
project-root/
  frontend/
    src/
      components/
        TopBar.jsx
        FilterBar.jsx
        NewsList.jsx
        EventCard.jsx
        MapView.jsx
        DetailPanel.jsx
        ActionPanel.jsx
        LoadingState.jsx
        ErrorState.jsx
      pages/
        Home.jsx
      store/
        useAppStore.js
      utils/
        formatEvent.js
        formatDate.js
        mapConfig.js
        categoryColors.js
    index.css
    main.jsx
  backend/
    src/
      server.js
      app.js
      routes/
        health.routes.js
```

```
        scan.routes.js
        events.routes.js
      services/
        gnewsService.js
        gemmaService.js
        firestoreService.js
        fallbackService.js
      utils/
        classifier.js
        coordinateMapper.js
        impactFormatter.js
        actionGenerator.js
        normalizeArticle.js
      data/
        fallbackNews.json
        regionMap.json
      validators/
        eventSchema.js
        articleSchema.js
      tests/
        classifier.test.js
        mapper.test.js
        formatter.test.js
        scan.route.test.js
  docs/
    technical_deep_dive_revised.tex
    non_technical_overview_revised.tex
  .env
  package.json
  README.md
```

## 15 Environment Variables

```
NODE_ENV=development
PORT=8080

GNEWS_API_KEY=your_key_here
GNEWS_BASE_URL=your_gnews_base_url_here

GOOGLE_CLOUD_PROJECT=your_project_id
VERTEX_LOCATION=us-central1
```

```
GEMMA_MODEL_ID=your_gemma_model_id

FIRESTORE_PROJECT_ID=your_project_id

FRONTEND_BASE_URL=http://localhost:5173
BACKEND_BASE_URL=http://localhost:8080
```

# 16 Implementation Flow

## 16.1 User Journey

1. User opens the app.
2. User presses Quick Scan.
3. Frontend calls the backend scan route.
4. Backend requests live news from GNews.
5. If the API fails, backend loads fallback JSON.
6. Backend filters irrelevant articles.
7. Backend classifies relevant articles.
8. Backend assigns region coordinates.
9. Backend sends the event to Gemma for explanation.
10. Backend merges AI output with structured metadata.
11. Frontend renders map pins and cards.
12. User clicks an event.
13. Detail panel shows finance impact and suggested action.

## 16.2 Event Processing Flow

1. Raw article arrives.
2. Article is validated with schema.
3. Keyword classification runs.
4. Region detection runs.
5. Coordinate mapping runs.

6. Gemma reasoning runs.
7. Output is normalized.
8. Firestore stores the result.
9. UI receives final event object.

# 17 Testing Strategy

## 17.1 Unit Tests

Test the following functions:

- classifier logic,
- coordinate mapper,
- impact formatter,
- action generator,
- article normalizer,
- fallback loader.

## 17.2 Integration Tests

Test the following flows:

- scan route with live API success,
- scan route with live API failure,
- fallback dataset response,
- event detail rendering,
- map marker generation,
- AI output merge.

## 17.3 Smoke Tests

Before demo day, verify:

- app opens,

- Quick Scan works,
- at least one event appears,
- map markers render,
- detail panel opens,
- personal finance impact is visible,
- suggested action is visible.

# 18 Deployment Plan

## 18.1 Local Development

Run frontend and backend locally with separate processes. The frontend talks to the backend through environment variables.

## 18.2 Cloud Deployment

Deploy the backend container to Cloud Run. Use Firestore for stored data. Use Vertex AI for Gemma requests. If needed, deploy the frontend separately or serve it as a static build from the same pipeline.

## 18.3 Demo Safety Rules

- Keep a fallback dataset inside the repo.
- Keep one offline demo path.
- Keep one preloaded screenshot or cached event set.
- Keep API keys in environment variables only.

# 19 Risks and Mitigation

| **Risk** | **Impact** | **Mitigation** |
|---|---|---|
| News API limits | Fewer live articles | Use local fallback JSON and caching. |
| Noisy headlines | Wrong classification | Use keyword scoring and whitelist rules. |

| | | |
|---|---|---|
| Gemma output errors | Broken explanation panel | Force JSON schema validation and fallback text. |
| Missing coordinates | No marker on map | Use country-level default coordinates. |
| Too much complexity | Slow build time | Keep the first version small and deterministic. |
| Weak finance linkage | Theme mismatch | Make personal finance impact mandatory. |
| Weak action layer | Feels like a normal summary tool | Add one suggested action to every event. |

## 20 Deliverables

The final prototype should include:

- a working React dashboard,
- a Quick Scan action,
- live headline ingestion,
- fallback data mode,
- a zoomable world map,
- clickable event markers,
- a detail panel with explanation,
- market impact,
- personal finance impact,
- suggested actions,
- tests for the core logic,
- and a deployable Cloud Run backend.

## 21 Conclusion

GeoFinance Lens is designed to be practical, visual, and buildable under hackathon constraints. The project should not try to predict the market. It should explain what the news means, where it happened, how it may affect the market, how it may affect personal money, and what the user should consider doing next.

That combination makes the app useful, understandable, and aligned with the theme.